Overview

This document expands the Creative Multi Solutions HR & Workforce Tracking System project plan into a detailed functional requirements specification and a full development plan with tasks, deliverables, and definitions of done. The original plan scoped the system as single-tenant; per instruction, multi-tenant support has been added as a first-class requirement across the data model, architecture, and every relevant development phase.[^1]

The system's core purpose is to give Creative Multi Solutions (and, with multi-tenancy, any client organizations layered on top of the platform) a single tool to manage employee KPIs, digital clock in/out, and real-time field location visibility for an outdoor, distributed workforce.[^1]

Functional Requirements (Detailed)

1. Multi-Tenant Organization Management

- The system must support multiple isolated tenant organizations (companies) on a single deployment, each with its own employees, KPIs, attendance data, and settings.
- Each tenant has a unique `tenant_id` used to scope every data query, API request, and real-time channel — no employee, KPI, attendance record, or location ping may ever be visible across tenant boundaries.
- Platform-level super-admins can create, suspend, or delete tenant accounts, and view aggregate system health across tenants (not tenant business data).
- Each tenant has at least one Tenant Admin role capable of managing that tenant's own employees, roles, KPIs, and settings independently.
- Tenant onboarding flow: create organization profile, invite/import employees, configure roles and KPI templates, set location-tracking policy (ping frequency, retention window).
- Tenant-level configuration: branding (logo/name on dashboard), working hours/timezone, notification preferences, data retention settings.
- Subdomain or workspace-based tenant resolution (e.g., `{tenant}.app.domain.com` or a tenant selector at login) so users land in the correct tenant context automatically.

2. Authentication & Access Control

- Secure login for employees, HR/tenant admins, and platform super-admins, scoped by tenant.
- Role-based access control (RBAC) with at least three roles: Employee, HR/Tenant Admin, Super Admin — each with distinct permissions.
- Password reset, session management, and token-based (JWT) authentication with tenant claims embedded in the token to enforce data isolation at the API layer.

- Support for BYOD authentication on mobile/PWA clients matching the same account across web and mobile.

3. Employee Management

- Create, update, deactivate employee profiles: name, role, contact info, assigned KPIs, device info, employment status (active/inactive), tenant association.
- Bulk import of employees (CSV) for tenant onboarding.
- Employee self-service profile view (read-only for most fields, editable for contact details).
- Search, filter, and list employees by role, status, or clock state within a tenant.

4. Digital Clock In/Out

- Employees clock in and out through web and/or mobile client with a single tap/click; system timestamps the event server-side (not client clock) to prevent manipulation.
- Each clock-in/out event captures a location snapshot (lat/lng) at that exact moment, independent of the continuous location stream.
- Duplicate clock-in prevention: an employee cannot clock in twice without clocking out first.
- Manual admin correction flow: HR/tenant admin can edit or annotate an attendance record (e.g., forgotten clock-out) with an audit trail of who changed what and when.
- Offline resilience: if a clock event is submitted while offline, it is queued locally and synced automatically once connectivity returns, preserving the original timestamp.
- Attendance history view per employee: list of clock-in/out pairs, computed shift duration, and flags for anomalies (e.g., missed clock-out, unusually long shift).

5. Live Location Tracking

- While clocked in, the employee's client streams location updates every 15-30 seconds to the backend via a real-time channel (WebSocket or managed pub-sub service).[^1]
- Location pings are buffered client-side and retried automatically if network connectivity drops, ensuring no gaps in the location trail once connectivity resumes.[^1]
- Each ping is tagged with `employee_id`, `tenant_id`, timestamp, and lat/lng coordinates.
- Location streaming stops automatically the moment an employee clocks out or the session times out.
- Configurable per-tenant ping frequency to balance battery/data usage against tracking granularity, within a system-defined min/max range.

6. Admin/HR Live Map Dashboard

- Real-time map view showing all currently clocked-in employees for the logged-in tenant, updated via the real-time channel with minimal latency.
- Map markers reflect current clock status (in/out) and last known location; clicking a marker shows employee name, role, and time since last update.
- Fallback to periodic REST polling for the map if the real-time layer is temporarily unavailable, so the admin view degrades gracefully rather than failing.[^1]
- Filter/search employees on the map by role, site, or status.
- Historical playback (optional v1.1 stretch): replay an employee's movement path for a given shift using stored clock-in/out location snapshots.

7. KPI & Performance Management

- Define KPIs per role or per employee, including KPI name, type (numeric/qualitative), target value, and review period (weekly/monthly/quarterly).
- Record periodic performance scores against each KPI, with reviewer notes and review date.
- Employee performance summary view: current scores vs. targets, trend over past periods.
- HR/tenant admin can create KPI templates that can be reused/cloned across employees or roles within their tenant.
- KPI due-date tracking that feeds the notification system (see below).

8. Notification System

- Automated notifications for: clock-in/out confirmation, missed clock-out (no clock-out recorded within X hours of clock-in), KPI review due, and admin-triggered announcements.[^1]
- Delivery via web push and/or mobile push (e.g., Firebase Cloud Messaging), scoped per tenant and per recipient.
- Notification inbox/list view with read/unread status; mark-as-read action.
- Admin-triggered broadcast announcements to all employees in a tenant, or to a filtered subset (by role).
- Notification delivery failure tracking as a monitored health metric.[^1]

9. Reporting & Basic Analytics

- Attendance summary reports per employee/team/tenant over a date range (hours worked, missed clock-outs, punctuality).
- KPI summary dashboard showing aggregate performance across a team or tenant at a glance.[^1]
- Export attendance and KPI summaries to CSV for offline reporting.

- Advanced analytics/BI reporting beyond basic summaries remains explicitly out of scope for v1.[^1]

10. Data Retention & Privacy Controls

- Raw location pings retained for a configurable window (default 30-90 days), then archived/aggregated or purged per tenant policy.[^1]
- Clock-in/out location snapshots retained long-term as part of attendance history, independent of the raw ping retention window.[^1]
- Tenant-level data export and deletion capability to support local labor-law and privacy compliance obligations (relevant given Egypt-based operations).[^1]

Non-Functional Requirements (Carried Forward)

Requirement	Detail
Performance	Lightweight, fast load times, low resource usage — especially the mobile clock-in flow [^1]
Scale	Support 50+ concurrent employees clocking in/reporting location simultaneously, designed for headroom up to 100 [^1]
Reliability	No lost location updates or clock events even with intermittent field connectivity [^1]
Multi-tenancy	Strict data isolation between tenants at database, API, and real-time channel levels (added requirement)
Data isolation	Every query and WebSocket subscription must be scoped by `tenant_id`; cross-tenant data leakage is a release blocker

Explicitly Out of Scope (v1)

- Payroll processing/integration, unless the client confirms it is needed.[^1]
- Advanced analytics/BI reporting beyond basic KPI summaries.[^1]
- Note: multi-company/multi-tenant support was originally listed as out of scope in the source plan, but per current direction it has been pulled into scope and is treated as a core requirement throughout this dev plan.[^1]

Full Development Plan

The build phase is organized into six phases following the Planning phase already defined in the source project plan. Multi-tenant support is embedded as a cross-cutting concern from Phase 1 onward rather than bolted on later.[^1]

Phase 1 — Foundations & Multi-Tenant Architecture (Weeks 1-2)

Tasks:

- Finalize tech stack (Node.js backend, PostgreSQL, WebSocket/managed pub-sub, PWA/React frontend, FCM) per the stack decision in the source plan.[^1]

- Design multi-tenant data model: add `tenant_id` as a foreign key on every core table (Employee, AttendanceRecord, LocationPing, KPIRecord, Notification).[^1]
- Decide tenant isolation strategy: shared database with row-level `tenant_id` scoping (recommended for this scale) vs. schema-per-tenant vs. database-per-tenant.
- Implement tenant provisioning service: create/suspend/delete tenant records, generate tenant-scoped admin accounts.
- Set up authentication with JWT tokens carrying `tenant_id` and role claims; implement middleware that enforces tenant scoping on every API request.
- Set up PaaS hosting environment, CI/CD pipeline, and environment configs (dev/staging/prod).

Deliverables:

- Multi-tenant database schema (migrations) with `tenant_id` scoping on all tables.
- Tenant provisioning API and basic super-admin console (CLI or minimal UI).
- Auth service with tenant-aware JWT issuance and validation middleware.
- CI/CD pipeline deploying to staging on every merge.

Definition of Done:

- A second test tenant can be created and its data is verifiably isolated from the first tenant (automated test proves cross-tenant queries return zero rows).
- All API endpoints reject requests missing a valid tenant-scoped token.
- CI pipeline runs lint, unit tests, and deploys to staging automatically.

Phase 2 — Core Employee, Attendance & Clock In/Out (Weeks 3-5)

Tasks:

- Build Employee CRUD APIs and admin UI, scoped per tenant, including bulk CSV import.
- Build `POST /v1/attendance/clock-in` and `/clock-out` endpoints with server-side timestamping and location snapshot capture.[^1]
- Implement duplicate clock-in prevention and offline queuing/retry logic on the client.
- Build attendance history views (employee self-service and admin) per tenant.
- Implement manual correction flow with audit trail for admins.

Deliverables:

- Employee management module (API + UI) with bulk import.
- Clock in/out flow functional on web and mobile-first PWA, tested under simulated network loss.
- Attendance history and correction UI.

Definition of Done:

- An employee can clock in/out from the PWA under intermittent connectivity and no event is lost (verified via forced network drop test).
- Admin corrections are logged with user, timestamp, and before/after values.
- All attendance data is correctly tenant-scoped in multi-tenant test scenarios.

Phase 3 — Real-Time Location Tracking & Live Map (Weeks 6-8)

Tasks:

- Implement WebSocket (or managed pub-sub) channel for continuous location streaming while clocked in.[^1]
- Implement client-side location buffering and retry-on-reconnect logic.[^1]
- Build `GET /v1/admin/live-map` / WebSocket feed for the admin dashboard, scoped so each tenant admin only sees their own tenant's employees.
- Implement REST-polling fallback for the map if the real-time layer degrades.[^1]
- Implement location ping retention/archival job (30-90 day rolling window).[^1]
- Load-test the real-time layer at 100 simultaneous connections per the scale target.[^1]

Deliverables:

- Live map dashboard showing real-time employee locations per tenant.
- Automated retention/archival job for location pings.
- Load test report confirming performance at target scale.

Definition of Done:

- Live map updates with under 5-second end-to-end latency for 100 concurrent simulated location streams within a single tenant, with no cross-tenant data appearing.
- Map dashboard falls back to REST polling within a defined timeout when the WebSocket layer is disconnected, without breaking the clock-in flow.
- Retention job correctly archives/purges pings older than the configured window per tenant policy.

Phase 4 — KPI & Performance Management (Weeks 9-10)

Tasks:

- Build KPI definition APIs (per role/employee) and KPI template cloning within a tenant.
- Build performance score recording and review-note capture.
- Build employee performance summary view and tenant-wide KPI summary dashboard.[^1]
- Wire KPI due-dates into the notification trigger list.

Deliverables:

- KPI management module (API + UI) with templates.

- Performance summary dashboards (employee and tenant-wide).

Definition of Done:

- A tenant admin can define a KPI, assign it to employees, record scores across at least two review periods, and see trend data reflected accurately in the summary view.
- KPI data for one tenant is never visible to another tenant in any view or API response.

Phase 5 — Notifications & Admin Dashboard Consolidation (Weeks 11-12)

Tasks:

- Integrate managed push notification service (FCM or equivalent) for web and mobile.[^1]
- Implement notification triggers: clock-in/out confirmation, missed clock-out, KPI review due, admin announcements.[^1]
- Build notification inbox with read/unread state and mark-as-read endpoint.[^1]
- Consolidate admin dashboard: unified view of live map, clock status, and KPI summaries at a glance.[^1]
- Implement basic monitoring for missed clock-outs, failed location syncs, and notification delivery failures.[^1]

Deliverables:

- Notification service fully wired to all defined trigger events.
- Consolidated admin/HR dashboard.
- Monitoring dashboard/alerts for the three key health metrics.

Definition of Done:

- All four notification types fire correctly in staging with delivery confirmed on both web and mobile test clients.
- Admin dashboard loads live map, clock status, and KPI summary for a tenant in a single view within acceptable load-time targets.
- Monitoring correctly flags a simulated missed clock-out and a simulated failed location sync.

Phase 6 — Multi-Tenant Hardening, QA & Launch Prep (Weeks 13-14)

Tasks:

- Full regression pass on tenant data isolation across every module (employees, attendance, location, KPIs, notifications).
- Security review: JWT tenant claim validation, RBAC enforcement, rate limiting on public endpoints.
- Performance/load testing at 100 concurrent users across at least two simultaneous tenants to confirm isolation holds under load.

- Finalize documentation: API contract (`/v1/` versioned endpoints), tenant onboarding runbook, data retention/privacy policy documentation.[^1]
- Client UAT (user acceptance testing) session and sign-off.

Deliverables:

- Multi-tenant security and isolation test report.
- Finalized API documentation and tenant onboarding runbook.
- UAT sign-off document.

Definition of Done:

- Zero cross-tenant data leakage found across all automated and manual test cases.
- Client has completed UAT and formally signed off on functional requirements as delivered.
- System sustains 100 concurrent simulated users across multiple tenants without degradation in clock-in latency or map update latency, matching the original scale target.[^1]

Cross-Cutting Definition of Done (Applies to Every Phase)

- All new API endpoints are tenant-scoped and covered by automated tests proving isolation.
- Code passes linting and unit tests in CI before merge.
- Feature has been demoed to the CTO lead and matches the functional requirement it implements.
- No known data-loss scenario exists for clock events or location pings under intermittent connectivity.[^1]
- Documentation (API contract or runbook) updated alongside the feature.

Open Items to Confirm With Client

The following items from the original plan remain open and should be resolved before or during Phase 1, now with multi-tenancy considerations added:

- Whether payroll integration is needed later, which affects data model decisions now (e.g., storing hourly rate).[^1]
- Acceptable location update frequency vs. employee battery/data usage trade-off, and whether this should be configurable per tenant.[^1]
- Device assumptions (BYOD vs. company-issued) affecting whether a PWA suffices or a native app is required.[^1]
- Data privacy expectations for continuous location tracking, including Egypt labor-law considerations, and whether privacy policies need to differ per tenant.[^1]
- Tenant onboarding model: will Creative Multi Solutions itself be the only tenant initially, with additional client organizations onboarded later, or is multi-tenancy needed immediately for parallel clients?

References

1. Project-Plan-Creative-Multi-Solutions-HR-system.pdf